

Setting up an OS PDK

FOSSI

Lawrence Quizon

Table of contents

- Steps Overview
- Resources
- Setting up LibreLane
 - Most important requirement: Config files
 - What's a TLEF?
 - Configuration Files Detail
 - Folder Structure
- Execution: Setting up XSchem
 - XSchem as the virtuoso equivalent
 - XSchem Symbols (like the Cadence symbol view)
- Execution: Setting up MAGIC
 - MAGIC setup checklist

Steps Overview

The following tools require setup for a new PDK:

- Yosys Synthesis
- OpenROAD ([place_opt](#), [global_route](#), [detailed_route](#), [cts](#))
- MAGIC setup (sets up DRC)
- NETGEN setup (LVS)
- OpenSTA settings
- OpenRCX settings
- XSchem (Symbol creation and spice attachment)
 - [PDK_HOME/libs.tech/xschem](#)

Resources

OpenLane Discussions Issue on the original repo discussing PDK setup:

<https://github.com/The-OpenROAD-Project/OpenLane/discussions/2133>

PDK Configuration Official documentation on setting up a configuration:

https://openlane.readthedocs.io/en/latest/reference/pdk_configuration.html

Setting up LibreLane

Most important requirement: Config files

In fact, little else matters because these config files point to everything that's needed. If they're not there, you have to make them.

Required Files:

1. **PDK Config:** `{pdk_root}/{pdk}/libs.tech/librelane/config.tcl`
2. **SCL Config:** `{pdk_root}/{pdk}/libs.tech/librelane/{scl}/config.tcl`

What's a TLEF?

You don't actually need it! The configuration file is where you declare the `tech lef`.

```
1 # Technology LEF
2 set ::env(TECH_LEF) "$::env(PDK_ROOT)/$::env(PDK)/libs.ref/$::env(STD_CELL_LIBRARY)/techlef/$::env(STD_CELL_
3 set ::env(TECH_LEF_MIN) "$::env(PDK_ROOT)/$::env(PDK)/libs.ref/$::env(STD_CELL_LIBRARY)/techlef/$::env(STD_C
4 set ::env(TECH_LEF_MAX) "$::env(PDK_ROOT)/$::env(PDK)/libs.ref/$::env(STD_CELL_LIBRARY)/techlef/$::env(STD_C
5 set ::env(CELLS_LEF) [glob "$::env(PDK_ROOT)/$::env(PDK)/libs.ref/$::env(STD_CELL_LIBRARY)/lef/*.lef"]
6 set ::env(GDS_FILES) [glob "$::env(PDK_ROOT)/$::env(PDK)/libs.ref/$::env(STD_CELL_LIBRARY)/gds/*.gds"]
7 set ::env(STD_CELL_LIBRARY_CDL) "$::env(PDK_ROOT)/$::env(PDK)/libs.ref/$::env(STD_CELL_LIBRARY)/cdl/$::env(S
```

Note: Except for `libs.ref` (which is standard), everything else is handled by this config file.

Configuration Files Detail

Source: [OpenLane Docs](#)

- `config.tcl`: Common info for all SCLs under this PDK. [Example](#)
- `common_pdn.tcl`: PDN configuration.
- `<scl>/config.tcl`: SCL specific info (overrides PDK config). [Example](#)
- `<scl>/tracks.info`: Metal layer offsets and pitches (extracted from tech lef). [Example](#)
 - Format: `<layer name> X|Y <offset> <pitch>`
- `<scl>/synth_exclude.cells`: Newline-separated cell names to trim during synthesis. [Example](#)
- `rcx_patterns.rules`: Rules for OpenSTA. [Example](#)

Folder Structure

- `<pdk_name>`
 - `libs.tech`
 - `openlane`
 - `config.tcl`
 - `common_pdn.tcl` (Default PDN script)
 - `<standard cell library>`
 - `config.tcl`
 - `tracks.info`
 - `no_synth.cells`
 - `drc_exclude.cells`
 - `libs.ref`
 - `skyxxx_fd_xx_xx`
 - `lef` (Can contain techlef declarations)
 - `techlef` (Doesn't have to be a folder)
 - `lib`
 - ...

Execution: Setting up XSchem

XSchem as the virtuoso equivalent

Note: The most basic way to start is skipping XSchem and using ngspice directly.

Checklist:

- Write `libs.tech/xschem/xschemrc`
- Create symbols for relevant elements
 - Suggest starting with `nfet` and `pfet` to try a flow.
 - Investigate `sky130` symbols (open with XSchem) for reference.

XSchem Symbols (like the Cadence symbol view)

You must create your own symbols (using XSchem).

You need to set global attributes for XSchem symbols for them to be parsed into a **spice** netlist properly.

Global attribute example () is shown on the right (model name: **nfet_1v8**)

```
1 1 G {}
2 2 K {type=nmos
3 3 lvs_format="@spiceprefix@name @pinlist sky1
4 4 format="@spiceprefix@name @pinlist sky130_f
5 5 + nf=@nf ad=@ad as=@as pd=@pd ps=@ps
6 6 + nrd=@nrd nrs=@nrs sa=@sa sb=@sb sd=@sd
7 7 + mult=@mult m=@mult"
8 8 template="name=M1
9 9 W=1
10 10 L=0.15
11 11 nf=1
12 12 mult=1
13 13 ad=\\ "expr('int((@nf + 1)/2) * @W / @nf * 0
14 14 pd=\\ "expr('2*int((@nf + 1)/2) * (@W / @nf
15 15 as=\\ "expr('int((@nf + 2)/2) * @W / @nf * 0
16 16 ps=\\ "expr('2*int((@nf + 2)/2) * (@W / @nf
17 17 nrd=\\ "expr('0.29 / @W ')\\ " nrs=\\ "expr('0
18 18 sa=0 sb=0 sd=0
19 19 model=nfet_01v8
20 20 spiceprefix=X
21 21 "
22 22 drc="fet_drc #{@name\\\\\\\\} #{@symname\\\\\\\\}
```

Execution: Setting up *MAGIC*

MAGIC setup checklist

Basically, all layout and signoff-related things (except maybe logic equivalence) are done by magic TCL scripts. Checklist:

- Process the given stdcell lefs to create `maglefs`
- Write `sky130A.tcl`
- Write `sky130A.tech`
- Write `sky130A.magicrc`
- Write all the custom TCL, like `fillgen`.

Note

I strongly recommend just copying then changing the fill and DRC scripts from the existing `sky130/libs.tech/magic` or `ihp-sg13g2/libs.tech/magic`